

COMICCRAFT

AI Comic Story Creator using Gemini Models

PROJECT TEAM

Role	Name	Register / ID
Team Leader	I.S Karan	07E73FEB CD676AFAC44BB6C6819C22
Member	S Boopathi Kumar	FB6A208A4D442BAB57E4B5B38688D152
Member	S.P BalaMurugan	FDB52655AFD65BAAC2C453B7E055B87C
Member	S.S Ahilash	51B16CFCFA1005C94094BB00055427B5

Project Domain: Artificial Intelligence & Generative AI

Technologies: Google Gemini Models, Python, FastAPI, Jinja2, HTML, CSS and JavaScript

1. Project Overview

ComicCraft is a Generative AI-based web application that creates comic stories from user-provided ideas or prompts. The application uses Google Gemini models to generate storylines, characters, dialogues, scene descriptions and comic-panel structures. The generated content is then presented through a dynamic web interface.

Project Objectives

- Build an AI-powered comic story creation platform.
- Use Gemini models for natural-language story generation.
- Accept simple user ideas and convert them into structured comic content.
- Generate characters, scenes, narration and dialogues.
- Provide an interactive and easy-to-use web interface.
- Demonstrate practical integration of Generative AI with a FastAPI application.

2. Prerequisites

Hardware Requirements

- Computer/Laptop
- Minimum 4 GB RAM
- Intel i3/Ryzen 3 processor or higher
- Internet connection
- Minimum 5 GB free storage

Software Requirements

- Python 3.10 or later
- Visual Studio Code / PyCharm
- FastAPI
- Uvicorn
- Jinja2
- Google Gemini API / current Gemini Python SDK
- HTML5, CSS3 and JavaScript
- Git/GitHub
- Modern web browser

Knowledge Requirements

Basic Python programming, HTML/CSS, JavaScript, REST APIs, JSON, Generative AI fundamentals and prompt engineering.

3. Research and Select the Appropriate Generative AI Model

The first development stage is to identify a Generative AI model suitable for creative story generation. Gemini models are selected for ComicCraft because the application requires natural-language

understanding and generation of structured creative content.

Selection Criteria

- Quality of generated stories
- Prompt understanding
- Context handling
- API accessibility
- Response performance
- Integration with Python
- Cost and usage considerations

Gemini Application

Gemini can generate comic titles, character descriptions, storylines, scene descriptions, dialogues, narration and panel-by-panel content based on user prompts.

4. Define the Architecture of the Application

ComicCraft follows a client-server architecture. The frontend collects the user's story idea, the FastAPI backend validates and processes the request, and the Gemini model generates the comic content. The resulting data is passed to Jinja2 templates for dynamic display.

Architecture Components

Frontend: HTML, CSS, JavaScript and Jinja2 templates.

Backend: Python, FastAPI and Uvicorn.

AI Layer: Google Gemini API.

Output: Comic title, characters, scenes, dialogues, narration and panels.

5. Workflow Diagram

Step	Component	Activity
1	User	Enters comic idea / prompt
2	Frontend	Collects and submits user input
3	FastAPI	Routes request and validates input
4	Prompt Processor	Builds structured Gemini prompt
5	Gemini Model	Generates story and comic content
6	Response Processor	Organizes generated response
7	Jinja2	Creates dynamic comic page
8	User	Views generated comic story

Workflow: User Prompt → Frontend → FastAPI → Prompt Processing → Gemini → Response Processing → Jinja2 → Final Comic

6. Set Up the Development Environment

Suggested Project Structure

```
ComicCraft/  
├── main.py  
├── requirements.txt  
├── .env  
├── templates/  
│   ├── index.html  
│   └── comic.html  
├── static/  
│   ├── css/  
│   └── style.css
```

- ■■■ js/
- ■■■ script.js
- generated/

Environment Setup

Create a Python virtual environment, install FastAPI, Uvicorn, Jinja2, python-dotenv and the appropriate current Gemini SDK, then configure the Gemini API key through an environment variable. The API key should not be hard-coded in source files.

7. Epic 2 – Core Functionalities Development

Core Functionalities

- User prompt input
- Prompt processing
- AI story generation
- Character generation
- Dialogue generation
- Scene and panel generation
- AI response processing
- Comic content display

Example User Prompt

“Create a short superhero comic about a college student who discovers an AI-powered robot.”

Expected AI Output

The system can generate a title, characters, storyline, scene descriptions, narration, dialogues and a structured sequence of comic panels.

8. Implement the FastAPI Backend to Manage Routing and User Input Processing

FastAPI acts as the main backend framework. It manages HTTP requests, validates user input, creates prompts, communicates with Gemini and sends the generated data to the frontend.

Main Routes

Method	Route	Purpose
GET	/	Display the home page
POST	/generate	Accept prompt and generate comic
GET	/comic	Display generated comic

Backend Processing Flow

User Request → FastAPI Route → Input Validation → Gemini Prompt → Gemini API → Response Processing → Jinja2 Template

9. Epic 4 – Frontend Development

Designing and Developing the User Interface

The frontend is designed to be simple, responsive and easy to use. It provides a clear story input area and controls for generating the comic.

Main UI Components

- ComicCraft title/logo
- Story idea text box
- Genre selector
- Character or story options
- Number of panels
- Generate Comic button
- Loading indicator
- Generated comic display

10. Creating Dynamic Templates with FastAPI's Jinja2

Jinja2 is used to dynamically insert AI-generated content into HTML pages. A single template can display different stories without creating a separate HTML file for each generated comic.

Dynamic Rendering Flow

Gemini Response → FastAPI → Jinja2 Context → Dynamic HTML → Comic Panels

The template can dynamically display the comic title, characters, panel number, scene, dialogue and narration.

11. Epic 5 – Testing and Deployment

Testing

Functional Testing: Verify prompt submission, Gemini response, panel generation and display.

API Testing: Test valid input, empty input, long input, API errors and network failures.

UI Testing: Check buttons, forms, loading states, responsive layout and comic display.

Error Handling: Provide meaningful messages for missing API keys, failed API requests, invalid input and usage limits.

Deployment

The application can be deployed on a cloud platform that supports Python/FastAPI applications. The project code can be maintained in Git/GitHub and the Gemini API key should be configured as a secure deployment environment variable.

Deployment Flow

Development → Local Testing → GitHub → Configure Environment Variables → Cloud Deployment → Production Testing

12. Future Enhancements

- AI-generated comic illustrations
- Multiple comic art styles
- Voice narration
- Multilingual comic generation
- User accounts and saved comics
- PDF/image export
- Personalized characters
- Story continuation and editing
- Advanced panel layout generation

13. Conclusion

ComicCraft – AI Comic Story Creator using Gemini Models demonstrates the integration of Generative AI with modern web development. The application accepts a simple user idea and converts it into structured comic content including characters, scenes, dialogues, narration and panels. By combining Gemini models with Python, FastAPI and Jinja2, the project provides a practical example of prompt engineering, API integration, backend development and dynamic frontend rendering.

The project also provides a foundation for future improvements such as AI-generated comic images, multilingual storytelling, voice narration and downloadable comic formats.

14. Team Roles

Name	Role	Primary Responsibility
I.S Karan	Team Leader	Project coordination, architecture, integration, testing and presentation

S Boopathi Kumar	Team Member	Core functionality and AI-related development
S.P BalaMurugan	Team Member	Backend/API and application development
S.S Ahilash	Team Member	Frontend, UI and testing support